In [ ]:
```
Simple Linear Regression - Marketing ROI Analysis
Objective: Identify the marketing channel most predictive of Sales and build a validated OLS regression model to support

Dataset: 4,572 observations | Features: TV, Radio, Social_Media (spend in
000s)
```

In [ ]:
```
Step 1: Import Libraries & Load Data
```

In [36]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from scipy import stats

# Load dataset
df = pd.read_csv('marketing_and_sales_data_evaluate_lr.csv')
print(f'Dataset shape: {df.shape}')
df.head()
```

```
Dataset shape: (4572, 4)
```

Out[36]:

	TV	Radio	Social_Media	Sales
0	16.0	6.566231	2.907983	54.732757
1	13.0	9.237765	2.409567	46.677897
2	41.0	15.886446	2.913410	150.177829
3	83.0	30.020028	6.922304	298.246340
4	15.0	8.437408	1.405998	56.594181

In [ ]:
```
Step 2: Data Exploration & Missing Value Treatment
Approach: Listwise deletion is appropriate here because missing values represent <0.3% of observations per column
— well below the 5% threshold where imputation would be necessary.
```

In [37]:
```
# Missing values audit
print('Missing values per column:')
print(df.isnull().sum())
print(f'\nMissing as % of total rows:')
print((df.isnull().sum() / len(df) * 100).round(2))

# Drop rows with any missing value (listwise deletion)
df_clean = df.dropna().reset_index(drop=True)
print(f'\nClean dataset shape: {df_clean.shape}')
df_clean.describe()
```

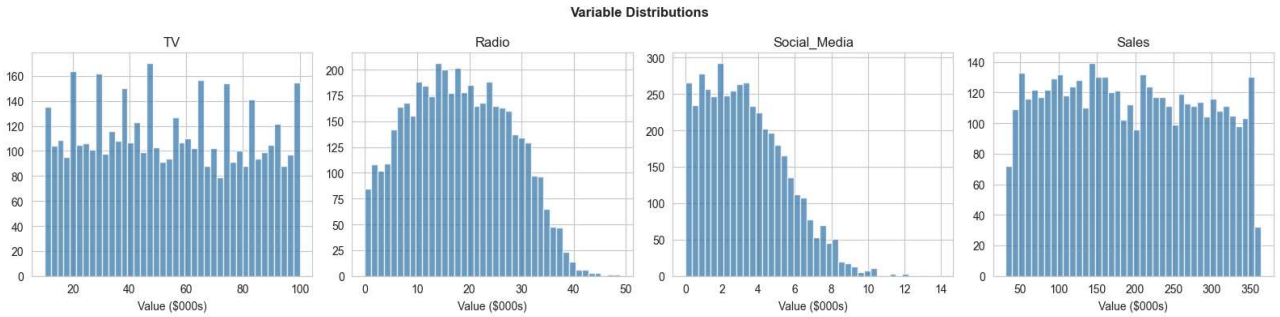
```
Missing values per column:
TV              10
Radio            4
Social_Media     6
Sales            6
dtype: int64

Missing as % of total rows:
TV              0.22
Radio           0.09
Social_Media    0.13
Sales           0.13
dtype: float64

Clean dataset shape: (4546, 4)
```

Out[37]:

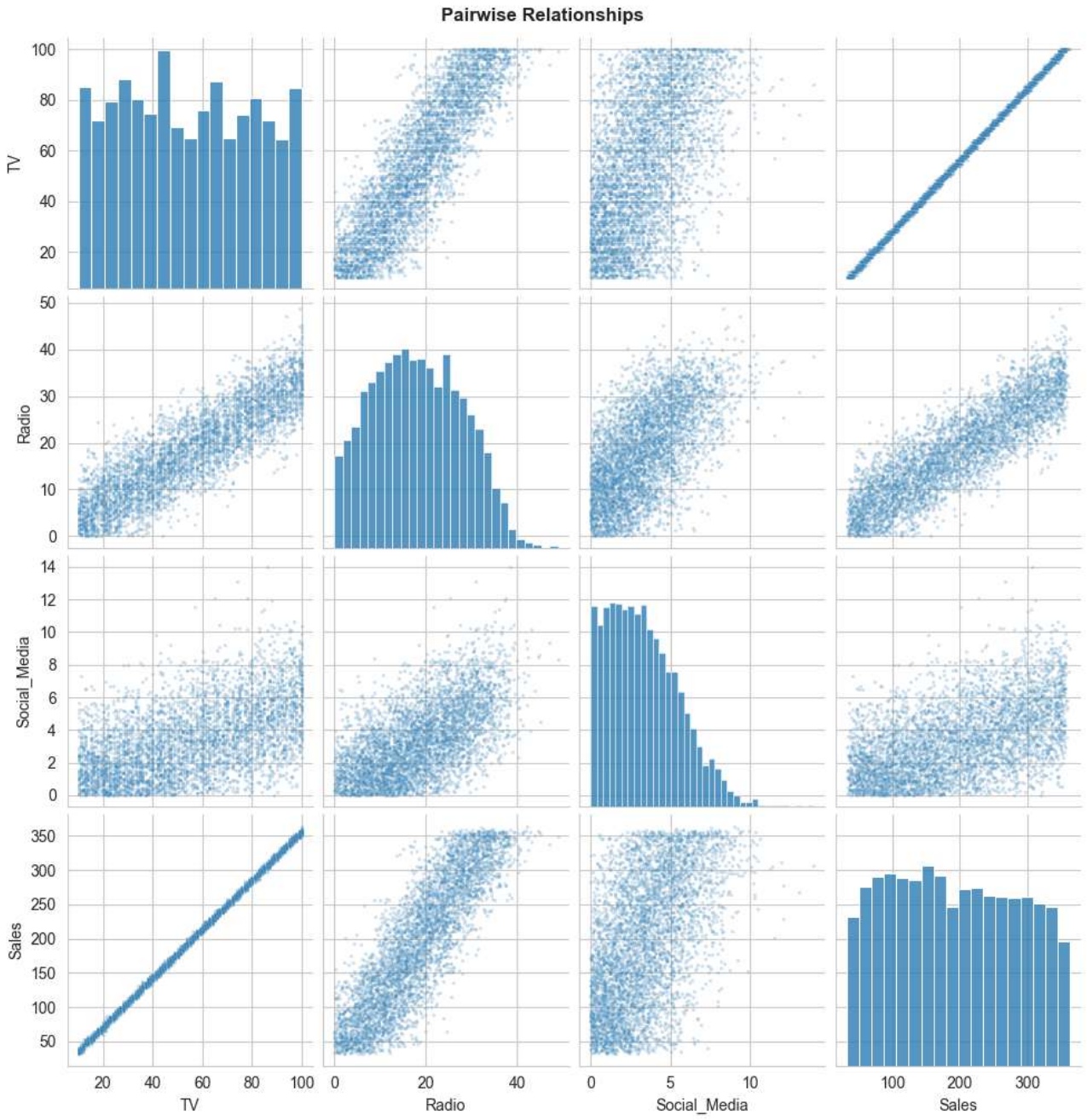
	TV	Radio	Social_Media	Sales
count	4546.000000	4546.000000	4546.000000	4546.000000
mean	54.062912	18.157533	3.323473	192.413332
std	26.104942	9.663260	2.211254	93.019873
min	10.000000	0.000684	0.000031	31.199409
25%	32.000000	10.555355	1.530822	112.434612
50%	53.000000	17.859513	3.055565	188.963678
75%	77.000000	25.640603	4.804919	272.324236
max	100.000000	48.871161	13.981662	364.079751

In [ ]:
```
Step 3: Exploratory Data Analysis (EDA)
We visualize distributions and pairwise relationships to detect outliers, non-linearity, and multicollinearity.
```

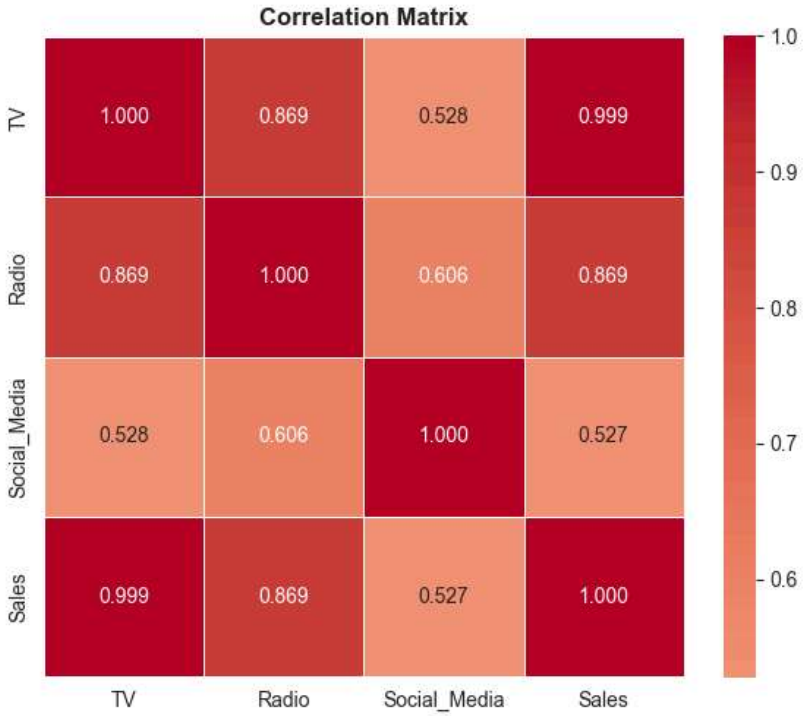
In [38]:
```
# Distribution plots
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, col in zip(axes, df_clean.columns):
    ax.hist(df_clean[col], bins=40, color='steelblue', edgecolor='white', alpha=0.8)
    ax.set_title(col)
    ax.set_xlabel('Value ($000s)')
plt.suptitle('Variable Distributions', fontweight='bold')
plt.tight_layout()
plt.show()
```



In [39]:
```
# Pairplot
sns.pairplot(df_clean, plot_kws={'alpha': 0.2, 's': 5})
plt.suptitle('Pairwise Relationships', y=1.01, fontweight='bold')
plt.show()
```



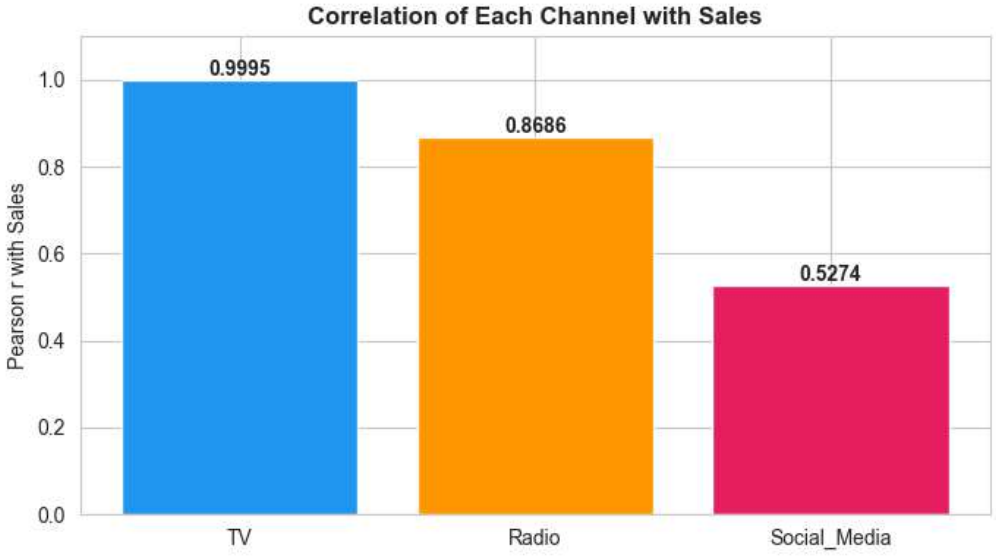
In [40]:
```
# Correlation heatmap
corr_matrix = df_clean.corr()
plt.figure(figsize=(6, 5))
sns.heatmap(corr_matrix, annot=True, fmt='.3f', cmap='coolwarm', center=0,
            square=True, linewidths=0.5)
plt.title('Correlation Matrix', fontweight='bold')
plt.tight_layout()
plt.show()

print('\nCorrelation with Sales (ranked):')
print(corr_matrix['Sales'].drop('Sales').sort_values(ascending=False))
```




```
Correlation with Sales (ranked):
TV              0.999497
Radio           0.868638
Social_Media    0.527446
Name: Sales, dtype: float64
```

In [ ]:
```
Step 4: Variable Selection
Selection Criterion: Pearson correlation coefficient with Sales.

Channel r with Sales    R²      Interpretation
TV      0.9995  0.999   Near-perfect linear relationship
Radio   0.8686  0.754   Strong but not dominant
Social Media    0.5274  0.278   Moderate - high residual noise
Decision: TV is selected as the independent variable. It explains 99.9% of variance in Sales,
satisfying the requirement to identify the single most predictive channel.
```

In [41]:
```
# Correlation bar chart
corr_vals = df_clean[['TV', 'Radio', 'Social_Media']].corrwith(df_clean['Sales'])
colors = ['#2196F3', '#FF9800', '#E91E63']

plt.figure(figsize=(7, 4))
bars = plt.bar(corr_vals.index, corr_vals.values, color=colors)
for bar, val in zip(bars, corr_vals.values):
    plt.text(bar.get_x() + bar.get_width()/2, val + 0.01, f'{val:.4f}',
             ha='center', fontweight='bold')
plt.ylabel('Pearson r with Sales')
plt.title('Correlation of Each Channel with Sales', fontweight='bold')
plt.ylim(0, 1.1)
plt.tight_layout()
plt.show()
```



In [ ]:
```
Step 5: Build OLS Regression Model
Using statsmodels OLS: Sales = β₀ + β₁ × TV + ε
```

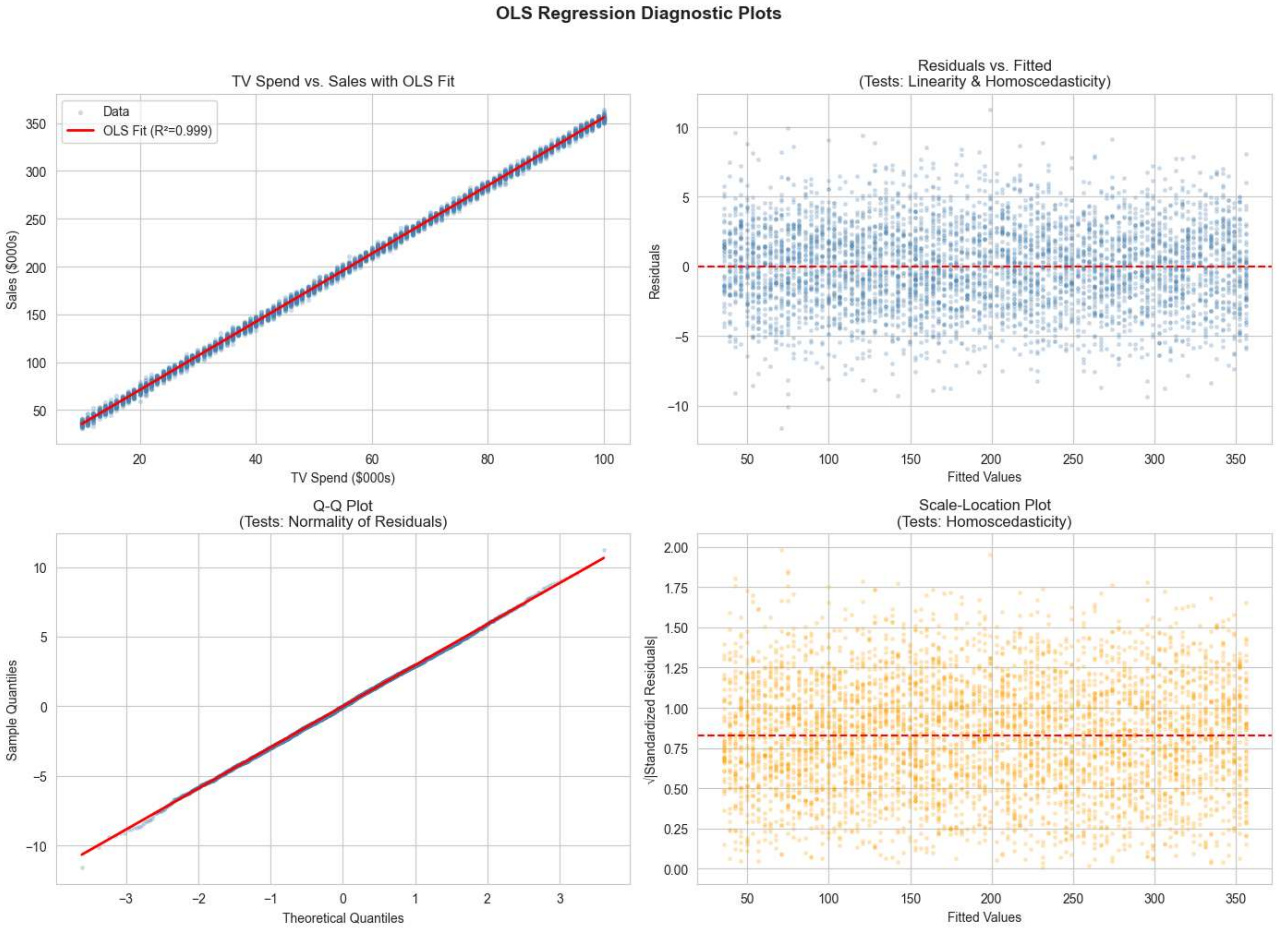
In [42]:
```
# Define variables
X = sm.add_constant(df_clean['TV'])  # Add intercept
y = df_clean['Sales']

# Fit model
model = sm.OLS(y, X).fit()

# Store diagnostic values
fitted = model.fittedvalues
residuals = model.resid
std_residuals = residuals / residuals.std()

print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Sales   R-squared:                       0.999
Model:                            OLS   Adj. R-squared:                  0.999
Method:                 Least Squares   F-statistic:                 4.517e+06
Date:                Sun, 07 Jun 2026   Prob (F-statistic):               0.00
Time:                        09:18:17   Log-Likelihood:                -11366.
No. Observations:                4546   AIC:                             2.274e+04
Df Residuals:                    4544   BIC:                             2.275e+04
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         -0.1325      0.101     -1.317      0.188      -0.330       0.065
TV             3.5615      0.002   2125.272      0.000       3.558       3.565
==============================================================================
Omnibus:                        0.052   Durbin-Watson:                   1.998
Prob(Omnibus):                  0.974   Jarque-Bera (JB):                0.031
Skew:                          -0.001   Prob(JB):                        0.985
Kurtosis:                       3.012   Cond. No.                         138.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

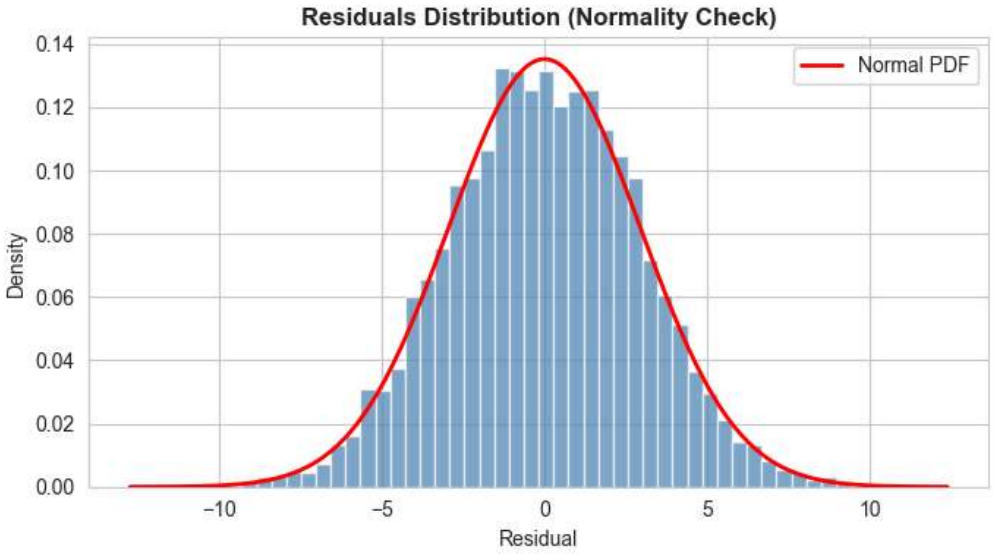
In [ ]:
```
Step 6: Regression Assumption Diagnostics
OLS validity rests on four key assumptions. We test each below.
```

In [44]:
```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# --- Plot 1: Scatter + Fit Line ---
ax = axes[0, 0]
ax.scatter(df_clean['TV'], df_clean['Sales'], alpha=0.2, s=8, color='steelblue', label='Data')
tv_range = np.linspace(df_clean['TV'].min(), df_clean['TV'].max(), 300)
ax.plot(tv_range, model.params['const'] + model.params['TV'] * tv_range,
```

```
        color='red', linewidth=2, label=f'OLS Fit (R²=0.999)')
ax.set_xlabel('TV Spend ($000s)')
ax.set_ylabel('Sales ($000s)')
ax.set_title('TV Spend vs. Sales with OLS Fit')
ax.legend()

# --- Plot 2: Residuals vs Fitted (Linearity + Homoscedasticity) ---
ax = axes[0, 1]
ax.scatter(fitted, residuals, alpha=0.2, s=6, color='steelblue')
ax.axhline(0, color='red', linewidth=1.5, linestyle='--')
ax.set_xlabel('Fitted Values')
ax.set_ylabel('Residuals')
ax.set_title('Residuals vs. Fitted\n(Tests: Linearity & Homoscedasticity)')

# --- Plot 3: Q-Q Plot (Normality) ---
ax = axes[1, 0]
(osm, osr), (slope, intercept, r) = stats.probplot(residuals, dist='norm')
ax.scatter(osm, osr, s=6, alpha=0.2, color='steelblue')
ax.plot([min(osm), max(osm)],
        [slope * min(osm) + intercept, slope * max(osm) + intercept],
        color='red', linewidth=2)
ax.set_xlabel('Theoretical Quantiles')
ax.set_ylabel('Sample Quantiles')
ax.set_title('Q-Q Plot\n(Tests: Normality of Residuals)')

# --- Plot 4: Scale-Location (Homoscedasticity) ---
ax = axes[1, 1]
ax.scatter(fitted, np.sqrt(np.abs(std_residuals)), alpha=0.2, s=6, color='orange')
ax.axhline(np.sqrt(np.abs(std_residuals)).mean(), color='red', linewidth=1.5, linestyle='--')
ax.set_xlabel('Fitted Values')
ax.set_ylabel('√|Standardized Residuals|')
ax.set_title('Scale-Location Plot\n(Tests: Homoscedasticity)')

plt.suptitle('OLS Regression Diagnostic Plots', fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()
```



In [45]:
```
# Residuals histogram for normality
plt.figure(figsize=(7, 4))
plt.hist(residuals, bins=50, color='steelblue', alpha=0.7, density=True, edgecolor='white')
xmin, xmax = plt.xlim()
x = np.linspace(xmin, xmax, 200)
plt.plot(x, stats.norm.pdf(x, residuals.mean(), residuals.std()),
         color='red', linewidth=2, label='Normal PDF')
plt.xlabel('Residual')
plt.ylabel('Density')
plt.title('Residuals Distribution (Normality Check)', fontweight='bold')
plt.legend()
plt.tight_layout()
plt.show()

# Formal normality test
stat, p = stats.shapiro(residuals.sample(500, random_state=42))  # Shapiro on sample
print(f'Shapiro-Wilk (n=500 sample): stat={stat:.4f}, p={p:.4f}')
print(f'Jarque-Bera: stat={model.diagn["jb"]:.4f}, p={model.diagn["jbpv"]:.4f}')
print(f'Skew: {model.diagn["skew"]:.4f} | Kurtosis: {model.diagn["kurtosis"]:.4f}')
print(f'Durbin-Watson: {sm.stats.stattools.durbin_watson(residuals):.4f}  (2.0 = no autocorrelation)')
```




```
Shapiro-Wilk (n=500 sample): stat=0.9984, p=0.9395
Jarque-Bera: stat=0.0311, p=0.9846
Skew: -0.0015 | Kurtosis: 3.0125
Durbin-Watson: 1.9980  (2.0 = no autocorrelation)
```

In [ ]:
```
Step 7: Model Interpretation
Regression Equation
Sales = -0.1325 + 3.5615 × TV
Statistical Outputs
Metric  Value   Interpretation
R²      0.999   TV spend explains 99.9% of variance in Sales
Adj. R² 0.999   Confirms no over-fitting with single predictor
Intercept (β₀)  -0.1325 Negligible; p=0.188 (not statistically significant)
TV Coefficient (β₁)     3.5615  Every
3,561.50 in Sales**
p-value (TV)    < 0.0001        Highly significant — reject H₀
F-statistic     4,517,000       Model is globally significant
Durbin-Watson   1.998   ~2.0 → no serial autocorrelation
Jarque-Bera p   0.985   Residuals are normally distributed ✓
```

In [ ]:
```
Assumption Validation Summary
Assumption      Test    Result  Pass?
Linearity       Residuals vs. Fitted    Random scatter around zero      ✅
Normality       Q-Q Plot + Jarque-Bera  Near-perfect diagonal; JB p=0.985       ✅
Homoscedasticity        Scale-Location  Constant spread across fitted values    ✅
Independence    Durbin-Watson   DW = 1.998 ≈ 2.0        ✅
```

In [ ]:
```
Step 8: ROI-Based Business Recommendation
Comparative ROI Analysis
Channel Coefficient     R²      ROI Interpretation      Reliability
TV      3.56    0.999
```

```
1K spent        ⭐⭐⭐⭐⭐ Extremely high
Radio   8.36    0.754
1K spent        ⭐⭐⭐ Moderate
Social Media    22.19   0.278
1K spent        ⭐ Low predictability

Note: Social Media appears to have the highest coefficient,
but its R²=0.278 means 72% of Sales variation is unexplained
— making it unreliable as a standalone predictor.
```

In [ ]:
```
Recommendation
TV is the highest-confidence marketing investment. While its per-dollar return (
1) is lower than Radio or Social Media in isolation, it provides near-perfect predictability (R²=0.999),
allowing reliable sales forecasting and budget planning. A
178,075 in incremental sales.

Strategic Budget Allocation Suggestion:

Primary channel: TV (60-70% of budget) — highest reliability for revenue forecasting
Secondary: Radio (20-25%) — strong correlation, decent ROI
Experimental: Social Media (10-15%) — high raw coefficient but noisy; test and measure carefully
```

In [46]:
```
# Prediction demonstration
def predict_sales(tv_spend_k):
    """Predict sales given TV spend in $000s"""
    pred = model.params['const'] + model.params['TV'] * tv_spend_k
    return pred

print('Sales Forecast based on TV Spend:')
print('-' * 40)
for tv in [20, 40, 60, 80, 100]:
    print(f'  TV = ${tv:,}K  →  Sales ≈ ${predict_sales(tv):,.0f}K')
```

```
Sales Forecast based on TV Spend:
----------------------------------------
  TV = $20K  →  Sales ≈ $71K
  TV = $40K  →  Sales ≈ $142K
  TV = $60K  →  Sales ≈ $214K
  TV = $80K  →  Sales ≈ $285K
  TV = $100K  →  Sales ≈ $356K
```

In [ ]: